Developer Knowledge Portal

Node.js Security & Compliance Guidelines

1. Purpose

The Developer Knowledge Portal exposes engineering knowledge through a centralized Node.js backend. This document defines the mandatory security controls that every backend service must implement. The objective is to provide consistent protection across React, Angular and React Native clients while maintaining predictable API behaviour.

The Developer Knowledge Portal exposes engineering knowledge through a centralized Node.js backend. This document defines the mandatory security controls that every backend service must implement. The objective is to provide consistent protection across React, Angular and React Native clients while maintaining predictable API behaviour.
The Developer Knowledge Portal exposes engineering knowledge through a centralized Node.js backend. This document defines the mandatory security controls that every backend service must implement. The objective is to provide consistent protection across React, Angular and React Native clients while maintaining predictable API behaviour.
The Developer Knowledge Portal exposes engineering knowledge through a centralized Node.js backend. This document defines the mandatory security controls that every backend service must implement. The objective is to provide consistent protection across React, Angular and React Native clients while maintaining predictable API behaviour.

2. Security Objectives

The platform prioritizes confidentiality, integrity and availability. Every API request is validated, logged and processed through standardized middleware. Services should follow the principle of least privilege, fail securely and expose only the minimum information required by clients.
The platform prioritizes confidentiality, integrity and availability. Every API request is validated, logged and processed through standardized middleware. Services should follow the principle of least privilege, fail securely and expose only the minimum information required by clients.
The platform prioritizes confidentiality, integrity and availability. Every API request is validated, logged and processed through standardized middleware. Services should follow the principle of least privilege, fail securely and expose only the minimum information required by clients.
The platform prioritizes confidentiality, integrity and availability. Every API request is validated, logged and processed through standardized middleware. Services should follow the principle of least privilege, fail securely and expose only the minimum information required by clients.

3. API Security Policy

All API endpoints serving the Knowledge Portal must enforce a strict CORS policy and require an 'X-Client-Version' header on every GET request. Requests missing this header must be rejected with an appropriate client error. API versioning follows the /api/v1 convention to support backward compatibility.

All API endpoints serving the Knowledge Portal must enforce a strict CORS policy and require an 'X-Client-Version' header on every GET request. Requests missing this header must be rejected with an appropriate client error. API versioning follows the /api/v1 convention to support backward compatibility.
All API endpoints serving the Knowledge Portal must enforce a strict CORS policy and require an 'X-Client-Version' header on every GET request. Requests missing this header must be rejected with an appropriate client error. API versioning follows the /api/v1 convention to support backward compatibility.
All API endpoints serving the Knowledge Portal must enforce a strict CORS policy and require an 'X-Client-Version' header on every GET request. Requests missing this header must be rejected with an appropriate client error. API versioning follows the /api/v1 convention to support backward compatibility.

4. Authentication & Authorization

Protected resources require JWT-based authentication. Tokens must be validated before business logic is executed. Role-based authorization determines access to administrative operations. Expired or malformed tokens must never be accepted.
Protected resources require JWT-based authentication. Tokens must be validated before business logic is executed. Role-based authorization determines access to administrative operations. Expired or malformed tokens must never be accepted.
Protected resources require JWT-based authentication. Tokens must be validated before business logic is executed. Role-based authorization determines access to administrative operations. Expired or malformed tokens must never be accepted.
Protected resources require JWT-based authentication. Tokens must be validated before business logic is executed. Role-based authorization determines access to administrative operations. Expired or malformed tokens must never be accepted.

5. Input Validation

Every path parameter, query parameter and request body is validated. Framework names such as react, angular, vue, python, java, c and cpp are validated against supported values. Invalid input is rejected before reaching the service layer.

Every path parameter, query parameter and request body is validated. Framework names such as react, angular, vue, python, java, c and cpp are validated against supported values. Invalid input is rejected before reaching the service layer.
Every path parameter, query parameter and request body is validated. Framework names such as react, angular, vue, python, java, c and cpp are validated against supported values. Invalid input is rejected before reaching the service layer.
Every path parameter, query parameter and request body is validated. Framework names such as react, angular, vue, python, java, c and cpp are validated against supported values. Invalid input is rejected before reaching the service layer.

6. Secure Headers & Logging

Security headers should be enabled consistently. Structured logs capture request identifiers, response status, latency and security events while avoiding storage of secrets or personal information. Audit logs should support operational investigations.
Security headers should be enabled consistently. Structured logs capture request identifiers, response status, latency and security events while avoiding storage of secrets or personal information. Audit logs should support operational investigations.
Security headers should be enabled consistently. Structured logs capture request identifiers, response status, latency and security events while avoiding storage of secrets or personal information. Audit logs should support operational investigations.
Security headers should be enabled consistently. Structured logs capture request identifiers, response status, latency and security events while avoiding storage of secrets or personal information. Audit logs should support operational investigations.

7. Operational Security

Redis, databases and application configuration must be isolated through environment variables. Secrets are never committed to source control. Health checks, metrics and monitoring should identify unusual traffic patterns, authentication failures and latency regressions.
Redis, databases and application configuration must be isolated through environment variables. Secrets are never committed to source control. Health checks, metrics and monitoring should identify unusual traffic patterns, authentication failures and latency regressions.
Redis, databases and application configuration must be isolated through environment variables. Secrets are never committed to source control. Health checks, metrics and monitoring should identify unusual traffic patterns, authentication failures and latency regressions.
Redis, databases and application configuration must be isolated through environment variables. Secrets are never committed to source control. Health checks, metrics and monitoring should identify unusual traffic patterns, authentication failures and latency regressions.

8. Compliance Checklist

Before deployment verify that: strict CORS is enabled, X-Client-Version is enforced on every GET request, JWT validation is active, request validation is enabled, structured logging is configured, rate limiting is applied where appropriate and API documentation reflects the implemented behaviour.

Before deployment verify that: strict CORS is enabled, X-Client-Version is enforced on every GET request, JWT validation is active, request validation is enabled, structured logging is configured, rate limiting is applied where appropriate and API documentation reflects the implemented behaviour.
Before deployment verify that: strict CORS is enabled, X-Client-Version is enforced on every GET request, JWT validation is active, request validation is enabled, structured logging is configured, rate limiting is applied where appropriate and API documentation reflects the implemented behaviour.
Before deployment verify that: strict CORS is enabled, X-Client-Version is enforced on every GET request, JWT validation is active, request validation is enabled, structured logging is configured, rate limiting is applied where appropriate and API documentation reflects the implemented behaviour.